평가 기준을
설계와 실행 근거로 연결했습니다

설계 의도 → 구현 위치 → 검증 결과 순서로 확인하는 담당 영역 60점

10 점 배점

아키텍처 설계

전체 AI 구조
ISO/IEC 25010 고려

02–03 / 부록 11

15 점 배점

AI Agent 설계

State · Node · 분기 · 반복
업무에 맞는 Agentic 흐름

04 / 부록 12

15 점 배점

AI Agent 구현

모델의 도구 선택과 근거
상태 저장 · 재개 · 이벤트

05–06 / 10

20 점 배점

MCP 설계·구현

업무 조회 · 변경 · RAG
연결 호출 · 예외 처리

07–09 / 10

본편 10장 + 부록 3장 · 수치는 배점이며, 자체 평가 점수나 ISO 인증을 의미하지 않습니다.

아키텍처 구성과 설계 판단 근거

구성요소와 책임 분리

구분	실제 구성	담당 역할
검사 진입	Actions / CLI Semgrep · AST	정적 검사·변경 근거를 마스킹하고 HMAC으로 Backend에 접수한다.
웹 콘솔	Vue 3 REST / SSE	결과 조회·검토·승인 UI를 제공한다. SSE 알림 후 REST로 재조회한다.
Backend	Spring Boot Security · Outbox	인증·검사 접수·결과 검증을 맡고, 업무 데이터와 Outbox를 저장한다.
비동기 전달	Kafka 검사·결과·진행·로그	검사 접수와 장시간 분석을 분리하고, 승인 재개·결과·진행 로그를 전달한다.
Agent	FastAPI · LangGraph LLM API	근거 조사·리뷰·상태 재개를 맡는다. Red/Blue는 한 Agent의 내부 단계다.
MCP 서비스	business-mcp rag-mcp	업무는 Backend 내부 API로 접근하고, 문서 근거는 Qdrant에서 조회한다.
저장소	PostgreSQL Qdrant	업무 기록·checkpoint는 PostgreSQL, 문서 벡터는 Qdrant에 저장한다.

아키텍처 흐름과 설계 근거

평가 관점	설계 내용과 판단 근거	근거
업무 목표와 전체 AI 구조	코드 변경 → 정적 근거 → 정책·문서 대조 → 리뷰. 실제 수집·승인 여부는 미확인 사항으로 구분한다.	D, G M
비동기 처리와 결과 정합성	검사·Outbox를 함께 저장한 뒤 202를 반환한다. Kafka 결과를 Backend가 검증·저장하고 SSE로 갱신을 알려 Vue가 REST로 재조회한다.	I, K
승인과 상태 재개	결정·Outbox 저장 후 기존 scan.request로 전달한다. Agent가 영속 승인 상태를 재조회하고 같은 scanId의 checkpoint를 재개한다.	C, A G
실패와 중복 전달	멱등 처리와 유한 재시도를 적용한다. Outbox 최대 3회·Worker 2회·소비 초기+2회. 위치에 따라 실패 기록 또는 DLQ/DLT로 보낸다.	I, K A
인증과 데이터 경계	세션·CSRF / HMAC / Bearer를 분리한다. business-mcp는 업무 DB에 직접 쓰지 않는다. 승인 후 변경은 CSI 내부 이슈 생성으로 제한한다.	S, I C, M
품질과 검증 범위	품질 9특성을 요구사항 → 설계 → 검증에 연결한다. 샘플 통과와 성능·정확도의 정량 평가를 구분한다.	D, T

근거 D ARCHITECTURE.md I Intake.java K KafkaPipeline.java C ApiController.java S Security.java
G graph.py A app.py M business.py / rag.py T 테스트·통합 기록 · 전체 파일 경로와 예외 사항은 발표자 노트

품질 기준을 반영한 아키텍처 설계

핵심 품질 요구와 설계

품질 기준	설계 내용과 확인 범위	근거
기능 적합성	실제 Finding·정적 사실·인용 ID를 검증한다. 가짜 근거 거부와 샘플 검사를 확인했으나, 대표 데이터셋의 오탐·미탐은 미측정이다.	QA-01
성능 효율성	202 비동기 접수와 모델·도구 예산을 적용한다. 반복·입력·호출 제한은 테스트했으나, p95 지연·처리량·실제 비용은 미측정이다.	QA-02
신뢰성	Outbox·멱등성·checkpoint·유한 재시도 적용. 승인·중복·Kafka 장애의 실행 기록이 있다. 강제 종료 후 복구·대규모 부하는 미검증이다.	QA-05
보안성	인증 경계·마스킹·읽기 도구 allowlist를 둔다. 변조·익명·CSRF 거부를 확인했으나, 종합 공격 대응과 다중 조직 권한은 과제다.	QA-06
안전성	HITL·Backend 승인 재검증으로 쓰기를 통제한다. 승인 시 내부 이슈 1건·거절 시 0건을 확인했다. 자동 패치·대상 앱 실행·운영 배포는 하지 않는다.	QA-09

운영·확장 품질과 검증 범위

품질 기준	설계 내용과 확인 범위	근거
호환성	JSON 계약·Streamable HTTP로 서비스를 잇는다. 구버전 조회와 로컬 통합 기록은 있으나, 다양한 클라이언트·버전 조합은 미검증이다.	QA-03
상호작용 능력	상태·근거·불확실성을 구분해 표시한다. 화면·SSE·모바일 E2E 기록은 있으나, 사용자 과업·접근성 평가는 부족하다.	QA-04
유지보수성	규칙·프롬프트·업무 API와 테스트를 분리한다. Python 40개·Java 8개 회귀 테스트 통과. 변경 비용·전체 분기 커버리지는 미측정이다.	QA-07
유연성	Compose·환경 변수로 배포 설정을 분리한다. 로컬 빌드·기동 기록은 있으나, 환경별 재현·다중 Worker·고가용성은 미검증이다.	QA-08
검증의 해석 범위	구현·샘플 검증·미측정 항목을 구분한다. 단위 테스트 대역과 과거 라이브 통합을 나누며, 전체 품질 달성이나 ISO 인증으로 설명하지 않는다.	V1-V4

근거 QA-01-09: docs/ARCHITECTURE.md의 품질 요구사항·설계·코드·검증 대응표
V1·V2: 2026-10-21 단위/경계 테스트 / V3: 2026-10-19 통합 기록 / V4: 2026-10-20 변경 리뷰·E2E 기록

업무 근거의 부족 여부가 다음 행동을 결정합니다

STATE / 기억하는 업무 정보

검사·변경
event · findings · changes

근거
policy · references · comparison

조사 과정
action · observation · trace

실행·승인
budget · approval · decision

GRAPH / 고정 단계와 모델 선택 분리

검사 검증
validate_scan

→

근거 로드
load_findings

→

조사 필요 판단
select_analysis_tools

규칙 또는 변경 존재 → 모델 선택 루프

다음 도구 선택
plan_analysis

→

인자 검증 · 실행
execute_analysis_tool

→

관측 · 근거 확인
assess_evidence

근거 부족 / 도구 오류 → 재선택

종료 요청 + 근거 충족

Red / Blue 검토
정적 사실 + LLM 리뷰 검증

→

결과 발행
scan.result → Backend 저장

→

현재 Finding 분기
있음: HITL / 없음: 종료

조사할 규칙·변경이 없으면 루프를 건너뜁니다. Red / Blue는 한 Agent 안의 분석 단계입니다.

근거: agent/csi_agent/graph.py State / build_graph · planner.py missing_requirements

모델의 선택과 근거 인용을 실행 기록으로 확인합니다

실제 LOCAL_DEMO 실행 기록

모델 선택 5턴 · 리뷰 배치 2회 · gpt-4.1-mini

1. **get_privacy_policy**
   검사 접수 시점의 정책 조회
2. **compare_policy_with_findings**
   필드를 승인·제한 목록과 대조
3. **search_internal_privacy_policy**
   사내 정책의 조항 근거 확보
4. **search_security_guidelines**
   외부 참조집의 관련 근거 확보
5. **finish_analysis**
   필수 근거 확인 후 종료 · 로컬 제어

근거가 답변을 제한

정적 사실
규칙 · 위치 · 마스킹 근거

정책·문서
문서 ID · 조항 · 페이지

검증된 리뷰
Red / Blue · 미확인 사항

허용된 ID·인용만 수락

trace의 LLM / RULE_BASED / LLM_FALLBACK을 구분합니다. 고정 초기 조회는 모델 선택으로 세지 않습니다.

보고서는 보존하고, 승인된 후속 작업만 재개합니다

thread_id = scanId · 승인 결정은 메시지 내용만 믿지 않고 DB에서 다시 읽습니다.

1

보고서 발행 후 대기

검토 요청 → interrupt()
PostgreSQL checkpoint

2

검토자의 승인 / 거절

Backend 결정 + Outbox
동일 트랜잭션 저장

3

같은 토픽으로 전달

scan.request
APPROVAL_DECIDED

4

영속 상태 확인 · 재개

승인 상태 재조회
Command(resume)

APPROVED → 내부 이슈 1건

Backend 승인 재검증 · approval별 멱등 처리

REJECTED → 이슈 생성 없이 종료

코드 변경 · 외부 GitHub 이슈 · 운영 배포 없음

실패 경로도 분리해 무한 재시도를 막습니다.

Outbox

최대 3회 → QUEUED 검사 FAILED

Agent Worker

최대 2회 → scan.dlq

Backend 소비

초기 + 2회 → 원본 토픽 .DLT

검사 완료 ≠ 후속 작업 승인 완료 · DLQ 자동 재처리와 강제 종료 복구 시험은 남은 과제

근거: app.py · graph.py · ApiController.java · KafkaPipeline.java · verification.json

업무 도구와 문서 검색 도구의 책임을 나눴습니다

business-mcp

읽기 6개 + 변경 2개 / Backend 내부 API

고정 초기 조회	get_scan_context · get_scan_findings get_scan_changes
모델 선택 읽기	get_privacy_policy · get_scan_history compare_policy_with_findings
통제된 쓰기	request_security_review → 검토 요청 create_security_issue → 승인 후 이슈

rag-mcp

읽기 3개 + Resource / Qdrant

search_internal_privacy_policy

search_security_guidelines

get_guideline_details

MCP Resource

security://guidelines/{rule_id}

규칙별 실제 조항 · 원문 위치 반환

모델의 선택 목록 = 읽기 도구 6개 + 로컬 finish_analysis. 쓰기 도구는 고정된 승인 워크플로가 통제합니다.

근거: mcp_servers/business.py · rag.py · planner.py READ_TOOLS

MCP 호출은 인자·권한·예산 검증을 통과해야 합니다

모델은 도구를 제안하고, Harness와 Backend가 실행 범위를 제한합니다.

LLM 함수 선택
허용된 읽기 함수
도구명 + 인자

→

Harness 검증
allowlist · 인자 schema
scanId 주입 · 호출 예산

→

FastMCP 호출
Streamable HTTP
Bearer 인증

→

서비스 경계
business → 내부 API
rag → Qdrant

오류를 다음 관측으로 전달하되, 조사와 변경을 무제한 허용하지 않습니다.

허용되지 않은 인자
쓰기 함수·다른 scanId·잘못된 규칙
→ 호출 제안 거부 / fallback

MCP 오류 · 시간 초과
TOOL_ERROR를 다음 선택에 전달
→ 대안 조사 또는 유한 실패

반복 · 모델 장애
도구별 3회 / 선택 최대 12턴
→ 소진 도구 제외 · 모드 표시

annotation은 권한 검사가 아닙니다. readOnlyHint / destructiveHint + 실제 인증·인자·승인 검증을 함께 적용합니다.

근거: planner.py validate_action · graph.py tool/execute · mcp_servers/common.py · tests/test_agent_loop.py

업무 맥락은 확인 가능한 정책·코드 사실까지 반영합니다

정책 PDF 2종
사내 정책 / 외부 참조집

→

조항 · 페이지 추출
catalog.json · 47청크

→

로컬 벡터화
1024차원 · 문자 n-gram

→

Qdrant 검색
유형·규칙 필터 / cosine

→

출처 반환
문서 ID · 조항 · 페이지

필드가 존재한다는 사실과, 업무상 수집이 승인됐다는 판단을 구분합니다.

반영된 업무 근거

정책의 승인·제한 목록
필드 역할 · 정적 저장 호출
실습 모드 조건 · 코드 변경 사실

아직 연결되지 않은 맥락

실제 수집 목적 · 보유 기간
별도 승인 기록 · 운영 설정
런타임 업무 로그

결과에 드러내는 한계

INSUFFICIENT_EVIDENCE
별도 승인 기록 미확인
런타임 검증 미수행

RAG 출처는 제공된 사내 정책·외부 참조집입니다. OWASP 공식 원문 전체나 실제 업무 승인 시스템과 구분합니다.

근거: rag/catalog.json · mcp_servers/index.py / vector.py / policy_review.py · scripts/evidence_context.py

설계 문서·코드·실행 결과를 함께 제시합니다

담당 영역 종합
배점 60점

40 Python 테스트 통과 **8** Java 테스트 통과

2026-10-21 · 로컬 코드 검증
라이브 전체 시스템 재실행과 구분

평가 영역	확인할 증거	대표 위치 / 기록
아키텍처 설계	책임·승인·실패 경로 + 품질 추적표	QA-01-09 / Outbox·멱등 처리
AI Agent 설계	State·조건 분기·선택 반복의 코드 대응	graph.py / missing_requirements
AI Agent 구현	실제 모델 선택·근거·승인 재개 기록	2026-10-19 LLM trace / HITL
MCP 설계·구현	실제 연결·도구 설명·권한·예외 처리	MCP Resource / 거부·재선택 시험

남은 검증: 오탐·미탐 / p95 지연·비용 / 사용자 평가 / 프로세스 강제 종료·부하·복구

근거: ARCHITECTURE.md V1-V4 · AGENT_HARNESS.md · CHANGE_REVIEWS.md · verification.json

부록 A. ISO/IEC 25010의 9개 품질 특성 대응

모든 항목을 문서화하되, 구현된 통제와 아직 측정하지 않은 품질을 구분합니다.

ID	품질 특성	CSI의 설계 결정	검증 상태 / 남은 범위
QA-01	**기능 적합성**	정적 근거 · 인용 ID · 변경 연결 검증	샘플 통과 / 대표 정확도 미측정
QA-02	**성능 효율성**	비동기 접수 · 모델/도구 예산 제한	제한 테스트 / p95·비용 미측정
QA-03	**호환성**	JSON 계약 · Streamable HTTP · 구버전 조회	로컬 통합 / 호환 행렬 미검증
QA-04	**상호작용 능력**	상태·근거·불확실성 · SSE 후 재조회	화면 E2E / 접근성 평가 부족
QA-05	**신뢰성**	Outbox · 멱등성 · checkpoint · 유한 재시도	승인·장애 기록 / 복구 시험 부족
QA-06	**보안성**	세션 / HMAC / Bearer · 마스킹 · allowlist	거부 시험 / 종합 공격 평가 부족
QA-07	**유지보수성**	규칙·프롬프트·API 분리 · 회귀 테스트	48개 테스트 / 변경 비용 미측정
QA-08	**유연성**	Compose · 환경 변수 · 서비스별 배포	로컬 기동 / 환경·확장 시험 부족
QA-09	**안전성**	HITL · 승인 재검증 · 임의 실행 제한	실행 통제 / 정식 위해 분석 없음

2011판 사용성 → 2023판 상호작용 능력 / 이식성 → 유연성 / 안전성 추가. 적합성 인증을 뜻하지 않습니다.

부록 B. 평가 문구에서 코드와 테스트로 바로 이동

평가 문구	구현의 시작점	검증의 시작점
전체 AI 구조 · 품질 고려	docs/ARCHITECTURE.md Intake.java · KafkaPipeline.java	V1·V2 단위 검증 V3 Outbox·중복·장애 기록
State · 분기 · 반복 설계	agent/csi_agent/graph.py planner.py · harness.py	tests/test_agent_loop.py 종료 거절 · 오류 후 재선택
모델 선택 · 근거 · 재개	planner.py · analysis.py · app.py PostgresSaver / Command(resume)	test_review_quality.py AGENT_HARNESS.md / V3 HITL
MCP 연결 · 변경 · 예외	mcp_servers/business.py · rag.py common.py · graph.py tool()	annotation · 쓰기 제안 거부 실제 Resource / 토큰 거부 기록

테스트 대역의 통과와 라이브 LLM·MCP·DB 통합의 증거를 분리하여 보여 주세요.

검증 시점: V1·V2 2026-10-21 / V3 2026-10-19 / V4 2026-10-20 · 상세는 ARCHITECTURE.md

부록 C. 화면에서 평가 근거를 확인하는 시연 순서

실행 결과의 “있음”보다, 같은 scanId에서 근거와 상태가 이어지는지 보여 줍니다.

1. **검사 상세**
   Repository · SHA
   분석 모드 · 정책 버전
2. **취약점 상세**
   Agent 도구 선택 trace
   턴 · 관측 · LLM 여부
3. **근거 · 리뷰**
   정적 사실 / 인용 조항
   Red·Blue / 미확인 사항
4. **실행 로그**
   MCP 시작·완료
   선택·예산·보고서 저장
5. **승인 후 재개**
   동일 scanId 이어가기
   승인 이슈 / 거절 종료

질문을 받았을 때 사용할 설명

왜 Agent인가?

관측 결과로 다음 도구를 선택하고
필수 근거가 있어야 종료합니다.

왜 MCP인가?

업무 API와 문서 검색을
명시적 도구 계약으로 분리합니다.

어디까지 검증했나?

정적 사실·정책·승인 흐름까지.
실제 업무 목적과 운영 규모는 과제입니다.

시연 전 조건: 실제 완료 데이터·LLM trace·참조 문서 / HITL 시연에는 PENDING 요청 필요

화면 경로: 검사 이력 → 검사 상세 → 취약점 상세 → 실행 로그 → 승인 대기 · 실제 데이터 필요